Programming for Business (CL-2016)

LAB INSTRUCTOR
OWAIS HAMEED

Week2 Objectives

- Operators
- Literals
  - String
  - Boolean
- String Data Types
  - Slicing
  - Modifiers
  - Concatenation
  - Escape characters

Python Operators

- In Python programming, Operators in general are used to perform operations on values and variables.

- **Operators:** Special symbols like -, + , * , /, etc.
- **Operands:** Value on which the operator is applied.

Types:

Operators in Python

Type	Operators
Arithmetic Operators	+, -, *, /, %
Relational Operators	< , <= , > , >=, ==, !=
Logical Operators	AND, OR, NOT
Bitwise operator	&, |, ^, ~, <<, >>
Assignment Operator	= , += , -= , *= , %=

Arithmetic

• Addition	(+)
• Subtraction	(-)
• Multiplication	(*)
• Division	(/)
• Floor Division	(//)
• Modulus	(%)
• Exponentiation	(**)

Relational Operators

- a = 13
- b = 33

```
print(a > b)
print(a < b)
print(a == b)
print(a != b)
print(a >= b)
print(a <= b)
```

Chaining Comparison Operators

- Python allows you to chain comparison operators:

- Example
- x = 5

- print(1 < x < 10)

- print(1 < x and x < 10)

Logical Operators

- Logical operators are used to combine conditional statements:
- and
- or
- not

Assignment Operators

- Python Assignment operators are used to assign values to the variables.
- b += a
- b -= a
- b *= a
- b /= a

Identity Operators

- In Python, is and is not are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.
  - is (True if the operands are identical)
  - is not (True if the operands are not identical)

```
print(a is not b)

print(a is c)
```

Membership Operators

- Membership operators are used to test if a sequence is presented in an object:

Operator Description

- in (Returns True if a sequence with the specified value is present in the object) x in y
- not in (Returns True if a sequence with the specified value is not present in the object) x not in y

Example

Check if "banana" is present in a list:

- fruits = ["apple", "banana", "cherry"]
- print("banana" in fruits)

Check if "pineapple" is NOT present in a list:

- fruits = ["apple", "banana", "cherry"]
- print("pineapple" not in fruits)

Membership in Strings

- The membership operators also work with strings:

- Example
- text = "Hello World"

- print("H" in text)
- print("hello" in text)
- print("z" not in text)

Take Multiple Input in Python

SPLIT() Method

- x, y = input("Enter two values: ").split()

print("Number of boys: ", x)

print("Number of girls: ", y)

String Literals

Types of String Literals:

- Single-quoted strings –
- Double-quoted strings –
- Triple-quoted strings – generally used for multi-line strings or docstrings.

Boolean Literals

Types of Boolean Literals:

- True – Represents a positive condition (equivalent to 1).
- False – Represents a negative condition (equivalent to 0).

String Data Type

- Python Strings are arrays of bytes representing Unicode characters. In Python, there is no character data type, a character is a string of length one. It is represented by str class.

- Access :
- s[1]
- s[-2]

Slicing Strings

- You can return a range of characters by using the slice syntax.
- Specify the start index and the end index, separated by a colon, to return a part of the string.
- Example
- Get the characters from position 2 to position 5 (not included):
- b = "Hello, World!"
- print(b[2:5])

Slice From the Start

- By leaving out the start index, the range will start at the first character:
- Example
- Get the characters from the start to position 5 (not included):
- b = "Hello, World!"
- print(b[:5])

Slice To the End

- By leaving out the end index, the range will go to the end:

- Example
- Get the characters from position 2, and all the way to the end:

- b = "Hello, World!"
- print(b[2:])

Negative Indexing

- Use negative indexes to start the slice from the end of the string:
- Example
- Get the characters:
- From: "o" in "World!" (position -5)
- To, but not included: "d" in "World!" (position -2):
- b = "Hello, World!"
- print(b[-5:-2])

Check String

- To check if a certain phrase or character is present in a string, we can use the keyword in.

- Example
- Check if "free" is present in the following text:

- txt = "The best things in life are free!"
- print("free" in txt)

Check if NOT

- To check if a certain phrase or character is NOT present in a string, we can use the keyword not in.

- Example
- Check if "expensive" is NOT present in the following text:

- txt = "The best things in life are free!"
- print("expensive" not in txt)

String Concatenation

- To concatenate, or combine, two strings you can use the + operator.

- Example
- Merge variable a with variable b into variable c:

- a = "Hello"
- b = "World"
- c = a + b
- print(c)

Example

- To add a space between them, add a " ":

- a = "Hello"
- b = "World"
- c = a + " " + b
- print(c)

String Format

- As we learned in the Python Variables chapter, we cannot combine strings and numbers like this:
- Example
- age = 36
- #This will produce an error:
- txt = "My name is John, I am " + age
- print(txt)
- But we can combine strings and numbers by using f-strings or the format() method!

F-Strings

- F-String was introduced in Python 3.6, and is now the preferred way of formatting strings.
- To specify a string as an f-string, simply put an f in front of the string literal, and add curly brackets {} as placeholders for variables and other operations.

Example

- Create an f-string:

- age = 36
- txt = f"My name is John, I am {age}"
- print(txt)

Placeholders and Modifiers

- A placeholder can contain variables, operations, functions, and modifiers to format the value.
- Example
- Add a placeholder for the price variable:
- price = 59
- txt = f"The price is {price} dollars"
- print(txt)

A placeholder can include a modifier to format the value.

A modifier is included by adding a colon : followed by a legal formatting type, like .2f which means fixed point number with 2 decimals:

Example

- Display the price with 2 decimals:

- price = 59
- txt = f"The price is {price:.2f} dollars"
- print(txt)

- A placeholder can contain Python code, like math operations:

Example

- Perform a math operation in the placeholder, and return the result:

- txt = f"The price is {20 * 59} dollars"
- print(txt)

Escape Character

- To insert characters that are illegal in a string, use an escape character.
- An escape character is a backslash \ followed by the character you want to insert.
- An example of an illegal character is a double quote inside a string that is surrounded by double quotes:

Example

- You will get an error if you use double quotes inside a string that is surrounded by double quotes:
- txt = "We are the so-called "Vikings" from the north."

- To fix this problem, use the escape character \":

- Example

- The escape character allows you to use double quotes when you normally would not be allowed:

- txt = "We are the so-called \"Vikings\" from the north.“

  print(txt)

Escape Characters

- Other escape characters used in Python:

- Code Result
- \ Single Quote
- \\ Backslash
- \n New Line
- \r Carriage Return
- \t Tab
- \b Backspace
- \f Form Feed
- \ooo Octal value
- \xhh Hex value

FINAL LAB TASK

- Take two integer numbers as input from the user.
  - Perform the following operations and display the result:
    - Floor Division up to 3 decimal places, modulus and percentage.
    - Compare the two numbers using relational operators (>, <, ==,!=,>=,<=).
    - Use logical operators (and, or, not)
- String Data types
  - Perform the following operations and display the result.
    - Take your first name and last name as separate inputs.
    - Check whether your name is in or not?
    - Display first 3 characters Last 4 characters and String in reverse order.
    - Concatenate both variables with a space.
    - Place a different data type in between string and display with multiple operations.